5CCSAGAP - Large Group Project Report - Team Los Caballos

Alma Loeblich, Anna Rachkova, Armita Eslami Nazari, Conor Brennan, Gregory Ceremisin, Arnav Gupta, Kamyar Nadarkhanidinehkaboudi, Omar Kolashinac and Sofia Davis

Deployed app: click here
Github: click here

1. Introduction

We have designed a RAG system that aids doctors in diagnosing patients with respiratory related illnesses. By providing an LLM with specialised data of roughly 535 distinct respiratory illnesses and previous diagnoses made by the system per patient, a tailored diagnosis is made upon the input of patientID and a free form text of the experienced symptoms.

1.1 Tech Stack

Orchestration - Docker Compose

UI - Streamlit

Text embedder - Mistral

Image Processor - TorchXRayVision

Medical Knowledge store - Pinecone

Patient history store - Chromadb

Conversation logs - MongoDB

Patient ID database - PostgreSQL

LLM - Mistral

RAG orchestration - LangChain

Backend - Inngest

RAG evaluation - RAGAS

Metrics Logging - ClearML

1.2 Requirements

Our main requirements are as follows (other dependent libraries are automatically installed by pip).

fastapi, inngest, streamlit, pinecone, chromadb, pymongo, ragas, sqlalchemy, langchain (mistralai), torchXRayVision, clearml, pytest

2. Methodology

2.1 Data Prep

The data preparation pipeline was designed to make retrieval consistent, safe, and reproducible across services. We transformed the respiratory corpus into a standard JSONL format (data/respiratory_data.jsonl, 535 records), where each entry contains embedding-ready clinical text (page_content) plus structured metadata (diagnosis, recommendation, source). Ingestion is orchestrated by Inngest: the workflow first validates that the file exists and is non-empty, then embeds records in batches through the shared embedding service (Mistral mistral-embed, 1024 dimensions) and upserts vectors into Pinecone (respiratory-knowledge-v1, respiratory-namespace, cosine metric). To protect production data, the index is created only if missing and never reset during sync. At runtime, free-text symptoms and X-ray system notes are normalised into retrieval terms, embedded with the same service for vector-space consistency, and combined with patient-specific context from ChromaDB and SQL metadata. For imaging input, X-rays are normalised and resized to 224×224 before TorchXRayVision inference, ensuring stable downstream integration with the text RAG flow.

2.2 Embedding Service

The embedding service is a lightweight Flask API running as a dedicated Docker service, converting text into 1024-dimensional vectors using Mistral's mistral-embed model. Isolating this as a separate microservice was a deliberate architectural choice: centralising all embedding logic means every service in the system (the RAG pipeline, ChromaDB, and Pinecone ingestion) calls the same endpoint without duplicating API code, and the model can be swapped in one place if needed. The service exposes /embed for single inputs and /embed/batch for bulk processing, the latter being particularly important during ingestion where batching 90 records per request is far more efficient than individual calls.

2.3 LLM Integration

The LLM integration layer connects the RAG pipeline to the language model with MistralAIChat configured at temperature 0.3 to favour consistent, factual outputs. Three prompt templates are defined in `diagnosis_prompts.py`: symptom extraction parses patient input or X-ray system notes into a JSON list of conditions for Pinecone retrieval; diagnosis generation produces a ranked differential diagnosis with confidence levels and recommended next steps; and diagnosis explanation handles follow-up questions by reasoning over the stored diagnosis result without re-triggering retrieval.

2.4 Medical Knowledge Base and Pinecone Retrieval

Two separate vector stores have been utilised to provide the LLM agent with relevant patient-specific information to aid in diagnosis.

The primary medical knowledge base stores 535 respiratory disease records in Pinecone, a cloud-hosted vector database. Respiratory conditions were chosen as the medical domain after evaluating available datasets; the symptom–disease dataset provided clear structured mappings covering Bronchial Asthma, Pneumonia, Tuberculosis, Cystic Fibrosis, and the Common Cold. Each record is stored as a JSONL entry containing condition name, symptoms, and recommended tests.

A secondary store, implemented using ChromaDB, maintains retrievable context from patient-specific chat sessions.

Ingestion

The production ingestion pipeline (`setup_pinecone.py`, triggered via Inngest on startup) reads the dataset in batches, embeds each record's `page_content` field via the shared embedding service, and upserts the resulting vectors to Pinecone with their metadata. IDs are generated as SHA-1 hashes of the text content to prevent duplicate uploads. The index is created automatically if it does not exist, using cosine similarity as the distance metric.

Retrieval

At query time, the patient's symptom description is embedded and Pinecone retrieves the `top_k = 5` most similar records. This value was confirmed through RAGAS evaluation as the best balance of context precision and recall. A de-duplication step ensures the same condition cannot appear twice in results, as multiple dataset entries describe the same disease with slightly different phrasing.

With the knowledge base populated and retrieval mechanisms defined, the final component of the pipeline is the orchestration layer that ties these elements together. The next section details how this orchestration is implemented and how follow-up queries reuse prior diagnostic context.

2.5 RAG Orchestration

The `DiagnosisOrchestrator` class in `diagnosis_orchestration.py` serves as the single entry point for all UI queries. It routes symptom descriptions through the full

extraction–retrieval–generation pipeline, while follow-up questions are handled by a lightweight explanation chain that reuses the stored diagnosis from the current session.

At query time, extracted terms are embedded via the shared embedding service and used to query the `respiratory-knowledge-v1` Pinecone index, retrieving the top 5 records validated through RAGAS. X-ray findings from the image processing service are passed as `[System Note:]`-tagged strings, parsed by the extraction chain, and converted into retrieval terms so that the full pipeline runs consistently regardless of input type.

2.6 Image Recognition

2.6.1 Architecture

The image processing module runs as a standalone FastAPI microservice, decoupling the computationally expensive DenseNet inference from the RAG pipeline so neither blocks the other. Pathology results are injected as [System Note:] strings so the RAG pipeline handles text and imaging inputs through the same extraction chain, avoiding a separate multimodal code path.

2.6.2 Preprocessing & Normalisation

The pipeline normalises four input formats (float, uint8, 16-bit, DICOM) to TorchXRayVision's expected range rather than rejecting non-standard formats in order to accommodate images exported from different clinical systems without requiring clinicians to convert files before upload.

2.6.3 X-Ray Validation

A fast heuristic intensity pre-filter runs before any model call to avoid unnecessary inference cost on clearly invalid inputs. A Pixtral-12b LLM check then acts as a second gate. The LLM prompt is biased toward false positives as, in a clinical context, rejecting a valid X-ray and blocking diagnosis is a worse outcome than occasionally passing a false image to inference, which will simply produce low-confidence predictions.

2.6.4 Model Inference & CAMs

DenseNet121 produces sigmoid-normalised per-pathology probabilities, filtered at 0.5 confidence to surface only clinically significant findings. CAMs are computed server-side and returned as raw arrays rather than rendered images, keeping the service stateless and delegating presentation logic entirely to the UI, keeping the service stateless and avoiding unnecessary image encoding and transmission over the internal network.

2.7 Conversation Logging (MongoDB Service)

The system uses Redis and MongoDB in tandem for session data — Redis handles low-latency caching of active chat state, while MongoDB provides durable long-term storage for the full clinical audit trail. This separation avoids the risk of losing session history if Redis is flushed, without sacrificing the response speed that in-memory caching provides. MongoDB was chosen over a relational store as the variable nested structure of pathology results and chat messages maps naturally to documents without requiring schema migrations.

Two collections are maintained: analysis_results for image processing outputs and chat_logs for session conversations. Raw scan files are stored on the local container filesystem rather than in MongoDB, as persisting large binary files in a document store would inflate storage and degrade query performance — MongoDB holds only the metadata and result IDs.

3. Evaluation

3.1 Evaluation with RAGAS

The RAG pipeline was evaluated using **RAGAS**, a framework that employs an LLM-as-a-judge to objectively score the performance of retrieval-augmented systems. For this project, **Mistral-7B** was used as the evaluator to maintain consistency with the embedding and generation models.

3.1.1 Experimental Design and Hypotheses

Our evaluation aimed to validate the hypothesis that a medical RAG system requires strict grounding constraints and minimal post-processing of retrieved context to achieve clinical reliability. We performed an iterative experimental process:

1. **Baseline Configuration**: Retrieval of 5 chunks (top-k=5) with a prompt requesting a broad list of 3-5 possible conditions.
2. **Optimized Configuration**: Reducing the retrieval window (top-k=3) and implementing strict prompt engineering to mandate direct "Evidence Quotes" and limit suggestions to 1-3 conditions.

3.1.2 Evaluation Metrics

Four core metrics were tracked to assess the system's diagnostic accuracy:

- **Faithfulness**: Measures if the answer is derived solely from the retrieved context (preventing hallucinations).
- **Answer Relevancy**: Assesses how well the response addresses the patient's specific symptoms.
- **Context Precision**: Evaluates whether the most relevant medical documents are ranked highest in the retrieval results.
- **Context Recall**: Measures if the system retrieved all necessary information to reach the ground-truth diagnosis.

3.1.2 Results and Prompt Engineering

The system underwent significant refinement to address an initial low **Faithfulness** score (0.44), which indicated the model was hallucinating diagnoses not supported by the provided literature. By transitioning from a "creative" prompt to a "strict grounding" prompt—requiring direct citations for every claim—we achieved a significant performance jump.

Metrices	Faithfulness	Answer Relevancy	Context Precision	Context Recall
Initial (Baseline)	0.4463	0.7606	0.6030	0.7333
Final Version	0.8264	0.7659	0.6136	0.7333

RAGAS Metrics Before and After Optimization

Note: Faithfulness improved by ~85% by reducing the expected condition count from 5 to 3 at first and providing raw, uncleaned context to the evaluator, ensuring no evidence was "hidden" from the judge during scoring.

3.1.3 ClearML Tracking

To ensure a systematic and reproducible evaluation process, all experiments were integrated with **ClearML**.

- **Metric Logging**: Every execution of `eval.py` automatically reported the four RAGAS scalars to the ClearML dashboard.
- **Experiment Comparison**: This allowed the team to compare the impact of different prompt versions and retrieval parameters (such as changing top-k from 5 to 3) in real-time, visualizing the trade-offs between precision and recall.
- **Traceability**: Each run captured the specific environment variables and model configurations used, ensuring that our "Faithfulness" gains were verifiable.

3.1.4 Concrete Instances (Product in Action)

The system was tested against 15 realistic clinical scenarios. For example:

- **Query**: "I have a productive cough with green mucus, fever, and chest pain when breathing."
- **System Response**: Successfully identified **Pneumonia** with high confidence, citing the requirement for a Chest X-ray and Sputum Culture as recommended in the retrieved medical literature.

4. User Interface and Deployment

The AI Respiratory Diagnosis Assistant's user interface is designed as a high-fidelity clinical dashboard that bridges the gap between complex multimodal backend services and the clinician. The UI is built using the **Streamlit** framework, chosen for its ability to handle real-time data reactivity and integrate Python-based machine learning outputs seamlessly.

4.1 User Experience and Interface Design

The interface emphasizes a professional, low-fatigue environment for clinical use.

- **Aesthetic Framework**: Using a custom CSS styling defined by `styles.css`, the UI implements a soft warm gradient background and a glassmorphism effect for chat components to improve visual hierarchy and readability.
- **Dark Mode Compatibility:** The stylesheet includes a `@media (prefers-color-scheme: dark)` block that automatically adapts the interface when the clinician's operating system is set to dark mode. Colour values are defined as CSS custom properties, allowing the media query to override the full palette in one place without duplicating rules. Native browser elements such as inputs and scrollbars additionally receive `color-scheme: dark` to ensure consistent rendering across all components.
- **Navigation**: A persistent sidebar allows clinicians to switch between patients, view consultation history, or exit the current session.

4.2 Clinical Workflow & Intake Logic

To ensure data integrity and patient safety, the UI enforces a mandatory intake protocol.

- **Patient Identification**: Upon launch, an overlay prevents access until the user identifies a patient via three entry points: searching existing records in the PostgreSQL database, registering a new patient profile, or entering a non-persistent "Guest Mode".
- **Session Persistence**: The UI utilizes `st.session_state` and query parameters to maintain session continuity, ensuring that refreshing the browser does not result in data loss during an active consultation.

4.3 Functional Features

The UI acts as the orchestrator for the system's multimodal capabilities:

- **RAG-Driven Consultation**: The main chat interface allows clinicians to input symptoms; the UI then displays the AI's diagnosis alongside an "expandable" section labeled **Clinical Evidence**, which shows the specific medical text retrieved from the Pinecone knowledge base for transparency. If not enough information is provided the LLM prompts the user for information, on top of this results are unclear from the retrieved information the LLM will again ask for more information.
- **X-Ray Analysis Visualization**: When an X-ray image is uploaded, the UI triggers the `image-processing-service` and renders:
    - **Pathology Probabilities**: A ranked list of detected conditions (e.g., Pneumonia, Infiltration) with corresponding confidence progress bars.
    - **Diagnostic Heatmaps**: Class Activation Maps (CAMs) generated via Matplotlib, which overlay disease patterns directly onto the radiology image to assist in clinical validation.

4.4 Memory and Storage Integration

The UI coordinates multiple database layers to provide a comprehensive patient history:

- **Short-Term Context**: Managed via **Redis**, which caches LLM responses and stores recent chat messages to provide low-latency conversation flow.
- **Long-Term Memory**: The UI integrates with **ChromaDB** to retrieve "semantic memory" from previous patient visits, allowing the RAG pipeline to consider a patient's historical medical context.
- **Persistent Logging**: Every interaction is synchronized to **MongoDB** via a dedicated `db-service`, ensuring a permanent audit trail of clinical sessions.

4.5 Deployment Strategy

The application is architected as a microservices-based system, orchestrated via **Docker Compose**.

- **Containerization**: The UI is isolated in its own **Docker** container (Dockerfile.ui) using a python:3.10-slim base image to ensure a lightweight and reproducible production environment.
- **Environment**: For the final deliverable, the system is deployed on an **Oracle Cloud Ubuntu VM**, making the assistant accessible via a public-facing IP (http://130.162.172.14:8501).
- **Networking**: The UI communicates with backend services (e.g., image-processing-service, db-service) over an internal Docker bridge network, securing internal API calls from public exposure.

4.6 GitHub Actions and CI

For UI and integrations, we used a GitHub Actions CI workflow on a self-hosted runner to automatically validate changes on every push and pull_request. The default pipeline runs focused core tests (Inngest function/endpoint and Pinecone safety checks) plus asset sanity checks (Python compile checks and notebook structure validation). Two manual workflow_dispatch options are kept for heavier verification: external integration scripts (with secrets and Docker services) and full notebook execution for image-processing validation. This gave us fast feedback for routine updates while still supporting deeper checks when needed.

5. Conclusion

The project ran into a number of practical problems along the way. On the infrastructure side, GitHub Actions was not working out of the box on the KCL instance as there were no runners available, so CI

jobs sat in queue until a self-hosted runner was configured manually. This slowed down testing during an important period of development.

Midway through the project the team decided to migrate from OpenRouter to Mistral for both the LLM and embeddings. This was the right call for consistency across the pipeline but required changes across multiple services at once and caused some integration issues in the short term.

Communication within the team was not always consistent. It was often unclear what had been completed and what was still in progress, and not all members contributed equally throughout the project. Better defined check-ins and a clearer shared understanding of task status would have helped avoid some of the bottlenecks and last-minute integration work.

Working with a microservices architecture also meant that local development and debugging took more effort than expected, and differences between team members' operating systems occasionally caused environment issues that were not immediately obvious.

Despite these challenges, the project produced a working end-to-end system, and the experience of building it as a team under real constraints was valuable in itself.

6. Project Management

6.1 Task and Role Allocation

The development of the AI Respiratory Diagnosis Assistant was highly collaborative, with tasks distributed across the team based on individual expertise and project requirements. The core responsibilities were allocated as follows:

- **Alma Loeblich:** Served as the Deployment Lead, managing the transition to Oracle. Also co-led the RAGAS evaluation implementation and embedded the respiratory database and setup Pinecone. Collaborated on chromaDB implementation with Conor.
- **Anna Rachkova:** Managed the LangChain orchestration, specifically implementing three distinct LangChain instances, adding confidence checking, and configuring stops to request extra context from users as well as collaborating on Image Processing.
- **Armita Eslami Nazari:** Contributed to image processing and llm integration, implemented main features for conversation on the UI.
- **Conor Brennan:** Took ownership of the ChromaDB and SQL implementation and integration, managing the patient history vector store and semantic memory retrieval. Added features to the existing UI to handle patient specific chats.
- **Gregory Ceremisin:** Led the development of the TorchXRayVision image processing module and MongoDB service. Organised project scheduling through Trello board and shared Outlook calendar. Handled testing for image processing and mongoDB, debugging and the final integration with the user interface. Report writing on the above mentioned.
- **Arnav Gupta:** Managed the Inngest backend workflow implementation, while also co-leading the setup of GitHub Actions for automated testing and deployment. Led data prep pre-developement phase.
- **Kamyar Nadarkhanidinehkaboudi:** Developed the embedding service. Led the implementation of the RAGAS evaluation framework and ClearML integration to track and verify diagnostic accuracy. Authored comprehensive testing suites for the embedding service, evaluation pipeline, and end-to-end system to ensure technical stability.
- **Omar Kolashinac:** Created two advisory meeting powerpoints, added Project Management part of report and ensured all technical decisions were accurately recorded post-code freeze.
- **Sofia Davis:** Managed the ClearML integration for metrics logging and experiment tracking, provided the powerpoint for the first advisory meeting, and oversaw the image processing service connecting to MongoDB.

6.2 Task Scheduling and Cadence

The project adhered to a strict agile methodology to maintain momentum and accountability. The team established a regular meeting cadence, convening every Monday and Thursday for synchronization. Formal minutes were recorded for every session to track decisions and progress. The team used Kanban (appendix 7.1 figure 1) and a shared outlook calendar (appendix 7.1 figure 2) for agile project management.

To prevent bottlenecks, tasks were broken down into smaller granularities with strict deadlines of one week or less. As the project approached its final stages, a “code freeze” was implemented by the final Saturday night, allowing the team to transition focus entirely to integration, deployment, and documentation.

6.3 Risk Management and Mitigation

Several risks were identified and actively managed throughout the project lifecycle:

1. Deployment Delays: The risk of failing to deliver a public-facing application was mitigated by adopting an “MVP-first” deployment strategy. The team prioritized a stable Docker environment hosted on Hugging Face, led by a dedicated Deployment Lead (Alma), ensuring a live demo was available for the final advisory meeting.
2. Evaluation Incompleteness: To prevent the system from relying on unverified LLM outputs, the team prioritized the RAGAS evaluation framework early in the development cycle. By confirming metrics like Faithfulness and Context Recall before the final merge, the team ensured the system was measurable and reliable.
3. Scope Creep: Discussions regarding the continued use of Inngest threatened to expand the project scope unnecessarily. This was controlled by implementing an MVP freeze and strategically prioritizing core RAG improvements over backend workflow expansions, pending advisor confirmation.
4. Infrastructure Inconsistencies: As multiple microservices were developed in parallel, inconsistent Dockerfile structures emerged as a risk to integration. This was mitigated by assigning clear ownership to specific infrastructure tasks and enforcing standardized branch naming and cleanup protocols.

7. Appendix

7.1 Project Scheduling

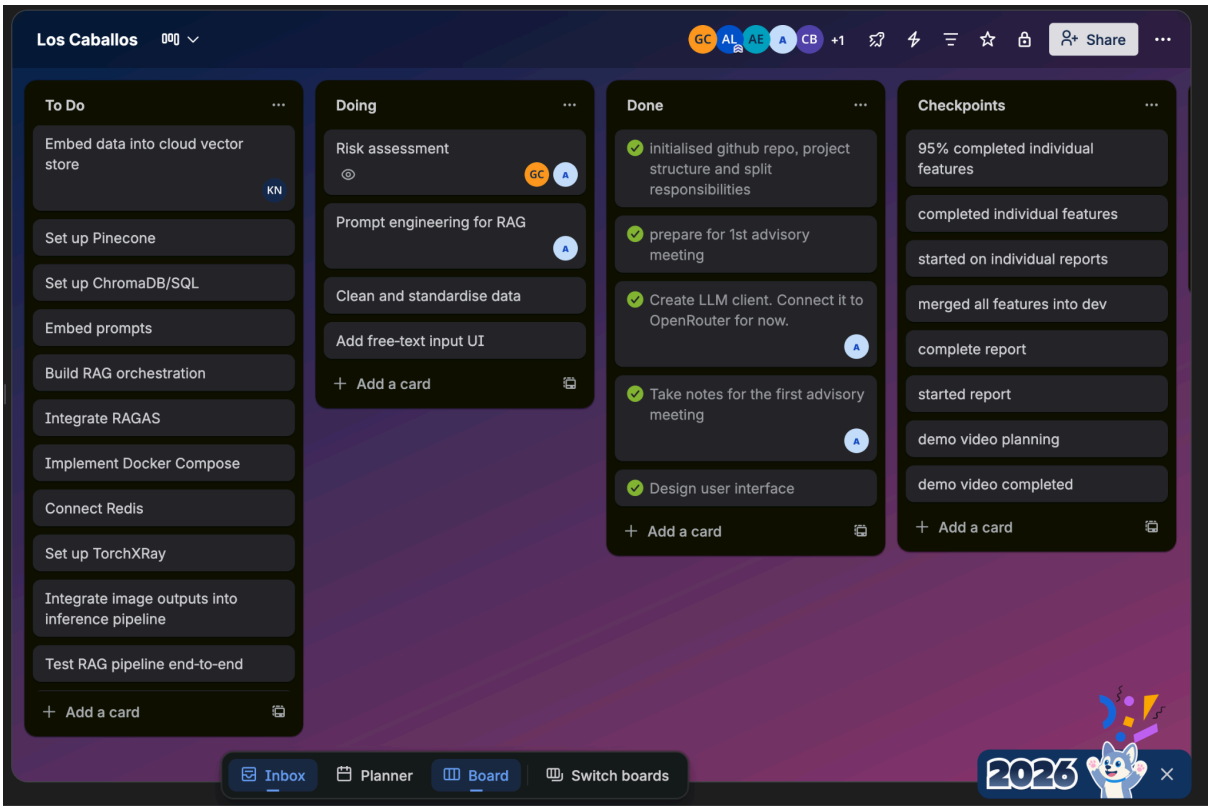


figure 1 - screenshot of the trello board used for kanban style scheduling

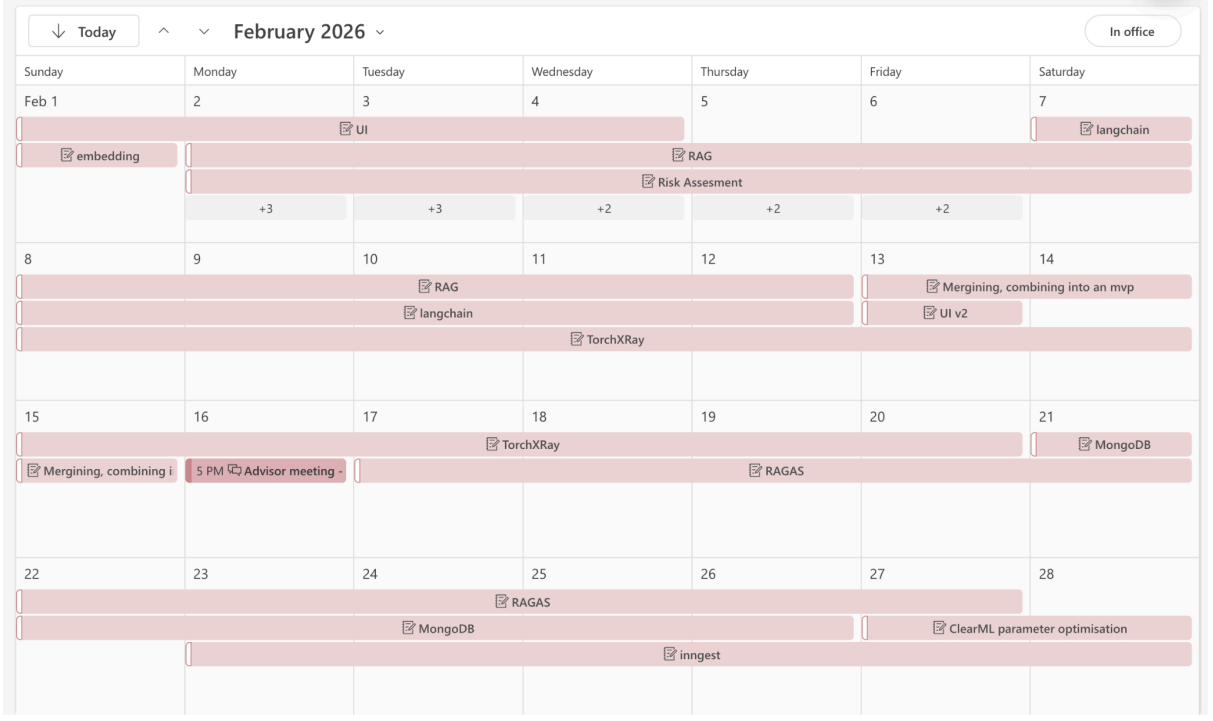


figure 2 - Screenshot of shared outlook calendar that was used for project scheduling